Week-7

1.1 Insert, delete and update data

After creating the structure of the table using DDL command CREATE. we can perform the following Operation into the Table such as

1) INSERT.
2) DELETE
3) UPDATE

INSERT.

The INSERT statement in SQL is used to insert the data into the table in database
It is possible to write the INSERT INTO statement in two ways:
1. Specify both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3, ).VALUES (value1, value2, value3, ...);
```

2. If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table. Here, the INSERT INTO syntax would be as follows:

```
INSERT INTO table_nameVALUES (value1, value2, value3, ...);
```

For Example; If the table structure is STUDENT (regno,name,address,sex)
Then insert statements will be

```
INSERT INTO STUDENT (regno,name,address,sex) values (121,'xyz','chanagara','M');
                    OR
INSERT INTO STUDENTvalues (121,'xyz','chanagara','M');
```

- Insert Data Only in Specified Columns

```
INSERT INTO STUDENT (regno,name,address) values (121,'xyz','chanagara');
```

In this case remaining attribute values will be NULL only.

Using SELECT in INSERT Statement:

We can use the SELECT statement with INSERT INTO statement to copy rows from one table and insert them into another table. The use of this statement is similar to that of INSERT INTO statement. The difference is that the SELECT statement is used here to select data from a different table. The different ways of using INSERT INTO SELECT statement are shown below:

- **Inserting all columns of a table:** We can copy all the data of a table and insert into in a different table.

```
INSERT INTO first_table SELECT * FROM second_table;
```

We have used the SELECT statement to copy the data from one table and INSERT INTO statement to insert in a different table.

- **Inserting specificcolumns of a table:** We can copy only those columns of a table which we want to insert into in a different table.
  Syntax:

```
INSERT INTO first_table(names_of_columns1) SELECT names_of_columns FROM second_table;
```

1.2 Modifying data: UPDATE and DELETE

UPDATE Statement:

The UPDATE statement in SQL is used to update the data of an existing table in database. We can update single columns as well as multiple columns using UPDATE statement as per our requirement.

Basic Syntax

```
UPDATE table_name SET column1 = value1, column2 = value2,...
WHERE condition;
```

Where
table_name: name of the table
column1: name of first , second, third column....
value1: new value for first, second, third column....
condition: condition to select the rows for which the values of columns needs to be update

For Example:

```
UPDATE STUDENT SET address='Chnagara' ,name='Vivek' WHERE regno=121;
```

In the above query the **SET** statement is used to set new values to the particular column and the **WHERE** clause is used to select the rows for which the columns are needed to be updated. If we have not used the WHERE clause then the columns in **all** the rows will be updated. So the WHERE clause is used to choose the particular rows.

DELETE Statement:

The DELETE Statement in SQL is used to delete existing records from a table. We can delete a single record or multiple records depending on the condition we specify in the WHERE clause.

Basic Syntax:

```
DELETE FROM table_name WHERE some_condition;
```

Where **table_name**: name of the table

some_condition: condition to choose particular record
we can delete single as well as multiple records depending on the condition we provide in WHERE clause..

For Example:

STUDENT

REGNO	NAME	ADDRESS	SEX
121	Sagar	Chnagar	M
122	vivek	Hassan	M
123	Gagan	Mysore	M

1)The above command will delete a single row with regno=121

```
DELETE FROM STUDENT WHERE regno=121;
```

2)If we omit the WHERE clause then all of the records will be deleted and the table will be empty

```
DELETE FROM STUDENT;
```

1.3 Update anomalies; impact of constraints.

There are different types of anomalies which can occur in referencing and referenced relation which can be discussed as:

STUDENT

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY	STUD_AGE
1	RAM	9716271721	Haryana	India	20
2	RAM	9898291281	Punjab	India	19
3	SUJIT	7898291981	Rajsthan	India	18
4	SURESH		Punjab	India	21

Table 1

STUDENT_COURSE

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

Table 2

Insertion anomaly: If a tuple is inserted in referencing relation and referencing attribute value is not present in referenced attribute, it will not allow inserting in referencing relation. For Example, If we try to insert a record in STUDENT_COURSE with STUD_NO =7, it will not allow.

Deletion and Updation anomaly: If a tuple is deleted or updated from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will not allow deleting the tuple from referenced relation. For Example, If we try to delete a record from STUDENT with STUD_NO =1, it will not allow. To avoid this, following can be used in query:

- **ON DELETE/UPDATE SET NULL:** If a tuple is deleted or updated from referenced relation and referenced attribute value is used by referencing attribute in referencing relation, it will delete/update the tuple from referenced relation and set the value of referencing attribute to NULL.
- **ON DELETE/UPDATE CASCADE:** If a tuple is deleted or updated from referencedrelation and referenced attribute value is used by referencing attribute in referencing relation, it will delete/update the tuple from referenced relation and referencing relation as well

1.4 Querying of available data: SELECT

SQL has one basic statement for retrieving information from a database: the**SELECT** statement.

Syntax:

```
SELECT <attribute list>
FROM <table list>
WHERE <condition>;
```

Where

■ <attribute list> is a list of attribute names whose values are to be retrieved by the query.
■ <table list> is a list of the relation names required to process the query.
■ <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

Q1. Retrieve All the Details of the employee(s)

```
SELECT * FROM EMPLOYEE;
```

This query involves only the EMPLOYEE relation listed in the FROM clause. The query selects the individual EMPLOYEE tuples which contains the Bdate and Address attributes listed in the SELECT clause.

Q2.**Retrieve the birth date and address of the employee(s) .**

```
SELECTBdate,addressFROM EMPLOYEE;
```

Q3. Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT Fname, Lname, Address
FROM EMPLOYEE, DEPARTMENT WHERE Dname='Research' AND
Dnumber=Dno;
```

1.5 Aliases.

In SQL, the same name can be used for two (or more) attributes as long as the attributes are in different relations. If this is the case, and a multi table query refers to two or more attributes with the same name, we must qualify the attribute name with the relation name to prevent ambiguity. We can also create an **alias** for each table name to avoid repeated typing of long table names.

Q1.For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn=S.Ssn;
```

In this case, we are required to declare alternative relation names E and S, called **aliases** or tuple variables, for the EMPLOYEE relation. An alias can follow the keyword AS, as shown in Q1, or it can directly follow the relation name—for example, by writing EMPLOYEE E, EMPLOYEE S as shown below

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE  E, EMPLOYEE  S
WHERE E.Super_ssn=S.Ssn;
```

1.6 sorting data: ORDER BY

- SQL allows the user to order the tuples in the result of a query by the values of one or more of the attributes that appear in the query result, by using the ORDER BY clause.
- The default order is in ascending order of values. We can specify the keyword DESC if we want to see the result in a descending order of values. The keyword ASC can be used to specify ascending order explicitly.

Syntax;

```
SELECT <attribute list>
FROM <table list>
[WHERE <condition>]
ORDER BY <attribute list> [ASC | DESC];
```

- The SELECT clause lists the attributes to be retrieved, and the FROM clause specifies all relations (tables) needed in the simple query. The WHERE clause identifies the conditions for selecting the tuples from these relations, including join conditions if needed. ORDER BY specifies an order for displaying the results of a query.

Q1. Retrieve All the Details of the employee(s) with decreasing order their salaries.

```
SELECT * FROM EMPLOYEE ORDER BY salary DESC;
```

Output:

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPER_SSN	DNO
James	E	Borg	888665555	10-NOV-31	450 Stone HouseTon TX	M	55000	-	1
Franklin	T	Wong	333445555	08-DEC-55	638 Voss HouseTon TX	M	55000	888665555	1
Jennifer	S	Walace	987654321	20-JUN-41	291 Berry Bellaire Tx	F	43000	888665555	4
Ramesh	K	Narayan	666884444	15-SEP-62	975 Free oak Humble Tx	M	38000	333445555	5
John	B	Smith	123456789	09-JAN-65	731 Fondren HouseTon TX	M	30000	333445555	5
Joyce	A	English	453453453	31-JUL-72	5631 Rice HouseTon Tx	F	25000	333445555	5
Ahmed	V	jabbar	987987987	10-NOV-37	980 dallas HouseTon Tx	M	25000	987654321	4
Alecia	J	Zeleya	999887777	19-JAN-68	3321 Caste Spring TX	F	25000	987654321	4

Q2. Retrieve All the Details of the employee(s) with alphabetical order of their Names.

```
SELECT* FROM EMPLOYEE ORDER BY Fname ;
```

Output:

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPER_SSN	DNO
Ahmed	V	jabbar	987987987	10-NOV-37	980 dallas HouseTon Tx	M	25000	987654321	4
Alecia	J	Zeleya	999887777	19-JAN-68	3321 Caste Spring TX	F	25000	987654321	4
Franklin	T	Wong	333445555	08-DEC-55	638 Voss HouseTon TX	M	55000	888665555	1
James	E	Borg	888665555	10-NOV-31	450 Stone HouseTon TX	M	55000	-	1
Jennifer	S	Walace	987654321	20-JUN-41	291 Berry Bellaire Tx	F	43000	888665555	4
John	B	Smith	123456789	09-JAN-65	731 Fondren HouseTon TX	M	30000	333445555	5
Joyce	A	English	453453453	31-JUL-72	5631 Rice HouseTon Tx	F	25000	333445555	5
Ramesh	K	Narayan	666884444	15-SEP-62	975 Free oak Humble Tx	M	38000	333445555	5

3. Retrieve All the Details of the employee(s) order by department no and salaries

```
SELECT * FROM EMPLOYEE ORDER BY dno,salary;
```

OUTPUT

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPER_SSN	DNO
Franklin	T	Wong	333445555	08-DEC-55	638 Voss HouseTon TX	M	55000	888665555	1
James	E	Borg	888665555	10-NOV-31	450 Stone HouseTon TX	M	55000	-	1
Alecia	J	Zeleya	999887777	19-JAN-68	3321 Caste Spring TX	F	25000	987654321	4
Ahmed	V	jabbar	987987987	10-NOV-37	980 dallas HouseTon Tx	M	25000	987654321	4
Jennifer	S	Walace	987654321	20-JUN-41	291 Berry Bellaire Tx	F	43000	888665555	4
Joyce	A	English	453453453	31-JUL-72	5631 Rice HouseTon Tx	F	25000	333445555	5
John	B	Smith	123456789	09-JAN-65	731 Fondren HouseTon TX	M	30000	333445555	5
Ramesh	K	Narayan	666884444	15-SEP-62	975 Free oak Humble Tx	M	38000	333445555	5